

Django 教學 01

開發環境與專案建立

第 1～2 章

從最小範例到專案實作與驗收

本份學習路線

先完成 Python 與 HTML/CSS 先備。

- **第 1 章：Python 與環境管理**
- **第 2 章：Django 專案骨架與啟動**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [來源索引](#) · [實作手冊對照](#)

第 1 章

Python 與環境管理

在 django_lab 建立環境，記下 Python、Django 版本與 interpreter 路徑。

本章的操作環境與成果

操作環境：自己的 django_lab；保留前章的 config 與 pages。

完成成果：在 django_lab 建立環境，記下 Python、Django 版本與 interpreter 路徑。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

1-1 先認識 uv：管理 Python 專案的工具

uv 幫我們安裝或選用 Python、建立虛擬環境、安裝套件，並記錄專案需要哪些套件版本。

- **Python** 執行程式，**Django** 提供網站框架。
- **uv** 準備執行環境，再呼叫 Python 或 Django 指令。
- 先完成環境準備，後面才會建立網站的程式檔案。

uv 準備環境 → Python 執行 Django → Django 處理網站功能

操作位置： 以下指令輸入終端機，不是 Python 的 `>>>` 視窗。

1-2 安裝 uv，再重新開啟終端機

macOS / Linux :

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows PowerShell :

```
winget install --id=astral-sh.uv -e
```

安裝後重新開啟終端機，執行 `uv --version`，應看到版本號。
若找不到 `uv`，先依安裝訊息檢查 PATH，再繼續下一步。

來源：[uv 安裝說明](#)

1-3 venv 與 `.venv/` 是什麼？

- **virtual environment (虛擬環境)**：隔離各專案使用的套件。
- **venv**：Python 內建的虛擬環境建立模組。
- `.venv/`：本課程放置虛擬環境的資料夾名稱。
- uv 也能建立虛擬環境，主線使用 `uv venv`。

練習 A/`.venv/` 各自安裝需要的 Django 版本
練習 B/`.venv/` 不必共用同一組套件

虛擬環境不是虛擬機，也不會替我們建立 Django 網頁或資料表。
沒有隔離時，在共用 Python 更新套件可能影響另一個專案。

1-4 Python：真正執行程式的直譯器

先用 uv 準備本課程的 Python：

```
uv python install 3.14.7
```

- **interpreter (直譯器)**：讀取並執行 Python 程式。
- **module (模組)**：可匯入或執行的 Python 程式單位。
- **package (套件)**：此處指可安裝使用的程式，例如 Django。
- **dependency (依賴)**：你的專案需要的套件。

即使電腦已有 Python，也要確認課堂指令使用哪個版本。
本次固定選用 3.14.7，方便全班依相同流程操作。

1-5 每次先確認「我在哪裡」

macOS/Linux：

```
pwd  
ls
```

PowerShell：

```
Get-Location  
Get-ChildItem
```

你應該看到 `manage.py` 與 `pyproject.toml`。

若看不到，不要繼續輸入 `migration/runserver` 指令。

1-6 建立自己的空練習資料夾

在自己存放課堂練習的位置開終端機，建立全新的資料夾：

```
mkdir django_lab
cd django_lab
```

終端機	顯示目前位置	列出檔案
macOS / Linux	<code>pwd</code>	<code>ls -a</code>
Windows PowerShell	<code>Get-Location</code>	<code>Get-ChildItem -Force</code>
Windows CMD	<code>cd</code>	<code>dir /a</code>

後續主線指令都在 `django_lab/` 執行。
把練習放在現有教材及完成專案之外，避免混用設定。

1-7 初始化 Python 專案與虛擬環境

```
uv init --bare --python 3.14.7 --vcs none
uv python pin 3.14.7
uv venv --python 3.14.7
uv run python --version
```

- **步驟 A** (`init --bare`)：建立最小的 `pyproject.toml`。
- **步驟 B** (`python pin`)：建立 `.python-version`，記下 Python 版本。
- **步驟 C** (`venv`)：建立 `.venv/`，存放這份練習使用的環境。
- **步驟 D** (`run`)：在專案環境執行 Python，應顯示 `Python 3.14.7`。

`--vcs none` 讓本次練習稍後再自行初始化 Git。

`uv run` 會準備、同步專案環境，也會產生需要的鎖檔。

1-8 確認執行的是哪一個 Python

```
uv run python -c "import sys; print(sys.executable)"  
uv run python -c "print('Python 環境準備完成')"
```

- `-c`：執行後面引號中的 Python 程式。
- 第一行應指向 `django_lab/.venv/` 裡的執行檔。
- 第二行應在終端機印出指定文字。
- `uv run` 後面接一般命令，不是另一套 Python 語法。

檢查點： 記下 Python 版本與執行檔位置。

若 VS Code 也要執行程式，選擇這個 `.venv` 的 interpreter。

1-9 uv run 的心智模型

```
uv run python manage.py check
```

拆開看：

```
uv run  
└── 用這個 project 的環境執行  
    └── python manage.py check
```

不要混用「系統 python」與專案環境中的 python。

1-10 安裝 Django，確認 django-admin 可用

```
uv add "django>=6.1.1,<6.2"  
uv run django-admin --version  
uv run python -m django --version
```

- `uv add` 安裝 Django，也更新依賴宣告及鎖檔。
- 版本條件加上引號，避免終端機把 `<`、`>` 當成特殊符號。
- `django-admin` 是安裝 Django 時一併提供的命令列工具。
- 正確名稱有連字號：`django-admin`，不是 `django admin`。
- 後兩行應顯示相同的 `6.1.x` 版本。

目前只裝好框架，還沒有 `manage.py`、project 或 app。

1-11 `python -m` 到底是什麼？

以下用 `pip` 說明語法，不在主線練習中額外執行。

```
python -m pip install django
```

- `python`：選定一個 Python interpreter。
- `-m pip`：請這個 interpreter 尋找並執行 `pip` 模組。
- `install django`：傳給 `pip` 的參數。

這種寫法明確指定「使用這一個 Python 所屬的 `pip`」，可避免直接輸入 `pip` 時，意外操作另一個 Python 的套件。

同理，`python -m django --version` 由指定 interpreter 執行 Django 模組。

1-12 傳統 venv：另一種環境準備方式

macOS / Linux (POSIX Shell)

```
python -m venv .venv  
source .venv/bin/activate  
python -m pip install "django>=6.1.1,<6.2"
```

Windows (CMD)

```
python -m venv .venv  
.venv\Scripts\activate.bat  
python -m pip install "django>=6.1.1,<6.2"
```

- 第一行建立 `.venv/`；第二行讓目前 shell / 終端機優先使用其中的 Python
- 第三行把 Django 安裝進目前 interpreter 的環境
- activation 只影響目前終端機工作階段

替代方式，不在 `django_lab` 重做。 此例假設 `python` 已可用；macOS 也可能使用 `python3`。離開環境用 `deactivate`。主線繼續使用 `uv run`，通常不必手動 activate。

1-13 三個環境檔案各管什麼？

<code>.python-version</code>	希望使用的 Python 版本（本練習：3.14.7）
<code>pyproject.toml</code>	專案直接宣告的需求與允許版本範圍
<code>uv.lock</code>	uv 解析後的完整、精確依賴結果

- 修改直接依賴：通常編輯 `pyproject.toml`，再讓 uv 更新鎖檔
- 不要把 `uv.lock` 當成手寫套件清單
- 不要提交 `.venv/`；其他人可由上述檔案重建

1-14 讀懂 pyproject.toml 的基本語法

練習檔案範例，版本範圍以 `uv add` 結果為準：

```
[project]
name = "django-lab"
version = "0.1.0"
requires-python = ">=3.14,<3.15"
dependencies = [
    "django>=6.1.1,<6.2",
]
```

`[project]` 是 TOML table；`key = value` 指定設定值。
字串用引號，`dependencies` 是字串陣列。TOML 是設定格式，
不執行 Python 程式，也不負責設定 Django 的資料庫。

1-15 版本範圍與鎖檔的精確版本

```
requires-python = ">=3.14,<3.15"  
django>=6.1.1,<6.2
```

- `>=3.14,<3.15`：允許 Python 3.14 系列
- `>=6.1.1`：允許 Django 6.1.1 以上
- `<6.2`：排除 6.2，避免跨入下一個 minor 系列
- 兩個條件以逗號連接，表示必須同時成立

`pyproject.toml` 寫「允許範圍」；`uv.lock` 會記錄目前實際解析到的精確版本。

1-16 直接依賴、開發依賴與同步

Django 是執行網站需要的直接依賴。Coverage 用來量測測試涵蓋率，可在需要時以 `uv add --dev "coverage>=7.6"` 加入開發群組。

```
[dependency-groups]
dev = ["coverage>=7.6"]

[tool.uv]
package = false
```

`package = false` 表示不把這份應用程式自身建成可發布的套件。

`uv sync` 依宣告與鎖檔同步 `.venv`，預設會包含 `dev` 群組。

虛擬環境隔離套件，鎖檔記錄精確依賴；重現仍需相容的系統與 Python。

1-17 uv sync 到底做什麼？

```
uv sync
```

它依 `pyproject.toml` / `uv.lock` 準備專案 Python 環境與依賴。

成功後不要只看「沒報錯」；再驗證：

```
uv run python --version  
uv run python -c "import django; print(django.get_version())"
```

1-18 ModuleNotFoundError 常見原因

ModuleNotFoundError: No module named '...'

檢查順序：

1. 有沒有 `uv sync`
2. 是否用 `uv run python ...`
3. app/module 名稱拼對嗎
4. `INSTALLED_APPS` 有嗎
5. import path 正確嗎

第 1 章實作與離堂檢核

任務： 在 django_lab 建立環境，記下 Python、Django 版本與 interpreter 路徑。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： LearnBoard 01 原第 1 章；LearnMart 01 原第 1 章。手冊保留原章號，對照表列出本課位置。

第 2 章

Django 專案骨架與啟動

建立 config 與 pages，確認 app 註冊、migrate、check 及歡迎頁。

本章的操作環境與成果

操作環境：自己的 django_lab；保留前章的 config 與 pages。

完成成果： 建立 config 與 pages，確認 app 註冊、migrate、check 及歡迎頁。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

2-1 先分清楚 project 與 app

Django project 集中網站共用的設定與入口。

Django app 集中一組相關功能，可以有自己的資料與頁面。

這份練習採用：

名稱	用途	即將產生的位置
django_lab	整份練習的外層資料夾	目前所在目錄
config	Django project package	config/
pages	第一個 app，用來練習頁面	pages/

一個 project 可組合多個 app。App 是功能模組，不必拆成微服務。

2-2 用 django-admin 建立 project

位置： `django_lab/` 。 **只執行一次：**

```
uv run django-admin startproject config .
```

- `uv run`：先使用課堂練習環境。
- `django-admin`：Django 命令列工具。
- `startproject`：產生網站設定與入口檔案。
- `config`：本次 project package 的名稱。
- `.`：把產物放在目前目錄，避免再多包一層資料夾。

檢查點： 目前目錄新增 `manage.py` 與 `config/`。
已建立後不重跑此指令，之後直接編輯檔案。

2-3 透過 manage.py 建立第一個 app

仍在 `django_lab/`，與 `manage.py` 同一層：

```
uv run python manage.py startapp pages
```

- `manage.py` 是剛才產生的專案命令入口。
- `startapp pages` 建立 `pages/` 及基本 Python 檔案。
- 不要先 `cd config` 或 `cd pages` 才執行。
- `startapp` 不會自動註冊 app，也不會建立 app 的 `urls.py`。

檢查點： `config/` 與 `pages/` 現在並排存在。

接著打開資料夾，逐一對照自己剛產生的檔案。

2-4 先看外層：每一項由誰建立？

django_lab/	
—— .python-version	uv python pin：Python 版本選擇
—— pyproject.toml	uv init / add：專案資訊與依賴
—— uv.lock	uv：解析後的精確套件版本
—— .venv/	uv：虛擬環境
—— manage.py	startproject：專案命令入口
—— config/	startproject：Django 設定 package
—— pages/	startapp：頁面功能 app

`uv init --bare` 不會產生一般 `uv init` 的 `main.py`、`README`。
`db.sqlite3` 稍後才在資料庫操作時出現，`.git/` 也尚未建立。

2-5 config：網站共用設定與入口

檔案	初學階段要知道的用途
<code>config/__init__.py</code>	標記一般 Python package，通常保留空白
<code>config/settings.py</code>	啟用哪些 app、資料庫、語言等設定
<code>config/urls.py</code>	網站最上層的網址分派表
<code>config/wsgi.py</code>	WSGI 伺服器載入 Django 的入口
<code>config/asgi.py</code>	ASGI 伺服器載入 Django 的入口

WSGI、ASGI 是 Python 網站與伺服器之間的介面。
這次練習先保留產生的入口檔案，後續部署再深入。

2-6 pages：功能程式從這裡開始

檔案／資料夾	用途
<code>__init__.py</code> 、 <code>apps.py</code>	package 標記、app 設定類別
<code>models.py</code>	定義資料模型
<code>migrations/</code>	保存資料庫結構變更紀錄，起初只有 <code>__init__.py</code>
<code>views.py</code>	接收 request，產生 response
<code>admin.py</code>	註冊後台可管理的 model
<code>tests.py</code>	自動檢查程式行為

`urls.py`、`forms.py`、`templates/` 不在 `startapp` 產物中，使用到時才自行新增。`static/` 放 CSS、JS 等資源，`media/` 放上傳檔案。

2-7 `manage.py` 如何知道要用哪份設定？

練習檔案：`manage.py` 的核心責任

```
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")  
execute_from_command_line(sys.argv)
```

- 指定 settings module 為 `config.settings`
- 把終端機參數交給 Django command 系統
- 因此 `runserver`、`migrate`、`test` 都知道要使用哪個專案設定

常見形式：

```
uv run python manage.py <command> [options]
```

2-8 Git：記錄你每次完成的修改

Git 保存檔案版本，方便比較修改與回到先前的狀態。
Repository（版本庫）是由 Git 管理的一組檔案與歷史。

在這份**獨立練習**的 `django_lab/` 中：

```
git --version  
git init  
git status
```

- `git init` 建立隱藏的 `.git/`，之後不用反覆執行。
- 若找不到 Git，先安裝 [Git](#) 並重新開啟終端機。
- Git 保存本機版本，GitHub 是可存放遠端版本庫的服務。
- `.git/` 是版本歷史，`config/` 是 Django package，兩者用途不同。

2-9 `.gitignore`：哪些檔案可以重建？

在 `django_lab/.gitignore` 新增：

```
.venv/  
__pycache__/  
*.py[cod]  
*.sqlite3  
/media/  
.env  
.DS_Store
```

保留程式、migration、templates、static、TOML、鎖檔與版本選擇檔。

虛擬環境、快取、本機資料庫與上傳檔可另行重建或備份。

`.env` 若含密鑰不可提交，Django 也不會自動讀取它。

2-10 保存第一個可比較的版本

確認 `git status` 沒有列出 `.venv/`，再執行：

```
git add .gitignore .python-version pyproject.toml uv.lock
git add manage.py config pages
git diff --cached
git commit -m "建立 Django 練習骨架"
```

- `add` 放入暫存區，`diff --cached` 檢查即將保存的內容。
- `commit` 建立版本紀錄。稍後編輯檔案可用 `git diff` 比較。
- 若 Git 要求姓名及 email，先以自己的資訊設定 `user.name`、`user.email`。
- `.gitignore` 不會自動取消已追蹤檔案，也不會刪除舊歷史中的密鑰。

2-11 settings 是 Python：先註冊自己的 app

打開 `config/settings.py`，**保留原有六個內建 app**，
在 `INSTALLED_APPS` 清單最後加上這一項：

```
"pages.apps.PagesConfig",
```

這個路徑指向 `pages/apps.py` 的 `PagesConfig`。
Django 也接受簡寫 `"pages"`，由 app 的預設設定進行發現。

接著修改同一份檔案的兩個值：

```
LANGUAGE_CODE = "zh-hant"  
TIME_ZONE = "Asia/Taipei"
```

`INSTALLED_APPS` 是 Python list，設定值是變數與字串。

2-12 settings 地圖：先知道要去哪裡找

設定	負責什麼
<code>BASE_DIR</code>	用 <code>Path</code> 找到外層練習資料夾
<code>INSTALLED_APPS</code>	啟用 app，影響 model、admin 等發現機制
<code>MIDDLEWARE</code>	request / response 經過的共用處理，例如 session、驗證
<code>ROOT_URLCONF</code>	根網址設定，本次是 <code>"config.urls"</code>
<code>TEMPLATES</code>	HTML 樣板引擎及搜尋位置
<code>WSGI_APPLICATION</code>	WSGI 應用入口

`asgi.py` 由 `startproject` 產生；完成專案另明列

`ASGI_APPLICATION = "config.asgi.application"`，骨架不一定有此設定。

2-13 settings 地圖：資料、語言與靜態資源

設定	負責什麼
DATABASES	資料庫引擎、位置及連線資訊，下一段實際操作
AUTH_PASSWORD_VALIDATORS	密碼驗證規則
LANGUAGE_CODE 、 TIME_ZONE	介面語言、預設時區
USE_I18N 、 USE_TZ	翻譯支援、具時區資訊的日期時間
STATIC_URL	CSS、JS 等靜態資源的 URL 前綴
DEFAULT_AUTO_FIELD	未明訂主鍵型別時採用的預設型別

後續需要共用靜態資料夾、上傳與登入功能時，才加入對應設定。
先能辨識用途，不必一次背完全部設定名稱。

2-14 App 註冊與 template 搜尋的差別

骨架的 `TEMPLATES` 預設包含 `"DIRS": []` 與 `"APP_DIRS": True`。
之後若自行建立外層 `templates/`，可把 `DIRS` 改為：

```
"DIRS": [BASE_DIR / "templates"],  
"APP_DIRS": True,
```

- `DIRS` 指定額外搜尋目錄。
- `APP_DIRS=True` 搜尋已安裝 app 裡的 `templates/`。
- 註冊 app 不等於建立資料夾，也不等於設定所有樣板路徑。
- 即使漏註冊 app，透過 `DIRS` 指定的根目錄樣板仍可能找到。

本次先用文字 response，不必提前建立 HTML 樣板。

2-15 本機開發設定的使用邊界

保留 `startproject` 產生的 `SECRET_KEY`，並在 settings 設定：

```
DEBUG = True  
ALLOWED_HOSTS = ["127.0.0.1", "localhost"]
```

- `SECRET_KEY` 參與簽章等安全機制，正式使用時須保密並外部化。
- `DEBUG=True` 提供除錯資訊，正式環境不能公開這些細節。
- `ALLOWED_HOSTS` 檢查請求的 Host，不是指定伺服器監聽位置。
- 正式部署需另外規劃伺服器、HTTPS、資料庫、備份及權限。

檢查點： 能指出 app 註冊、時區、URL 與開發設定的位置。

2-16 資料庫：持續保存網站資料

資料表 (table) 定義欄位，資料列 (row) 保存一筆內容。
Schema 指資料庫結構，包含資料表、欄位與限制。

`config/settings.py` 預設使用 SQLite：

```
DATABASES = {  
    "default": {  
        "ENGINE": "django.db.backends.sqlite3",  
        "NAME": BASE_DIR / "db.sqlite3",  
    }  
}
```

SQLite 使用本機檔案，適合課堂練習，不需先啟動獨立資料庫服務。

`uv add` / `uv sync` 安裝套件，不會替 Django 建好資料表。

2-17 第一次 migrate 前：先決定 User 的路線

本課練習 django_lab 使用 Django 內建 User，接著可執行第一次 migrate。

若未來的新專案需要自訂 User，應在第一次建立資料表前規劃。

現有 LearnMart 已預先建立自訂 User，並設定：

```
# 現有 LearnMart 的設定；不要貼入 django_lab  
AUTH_USER_MODEL = "marketplace.User"
```

第 16 章會解釋 AbstractUser、角色與帳號流程；現在先記住這項初始化決策。

取得現有專案時保留其 User 設定及 migration，依專案流程啟動。

2-18 第一次 migrate：建立內建功能的資料表

位置：`django_lab/`。

```
uv run python manage.py check
uv run python manage.py migrate
uv run python manage.py showmigrations
```

- `check` 檢查部分設定與程式結構問題。
- `migrate` 套用 migration，準備登入、session、admin 等內建資料表。
- `showmigrations` 的 `[X]` 表示已套用，`[ ]` 表示尚未套用。
- 此時尚未替 `pages` 定義 model，所以沒有自訂資料表。

檢查點：看見 `db.sqlite3`，migration 套用成功。
不要用刪除 migration 檔案的方式處理不懂的錯誤。

2-19 分清楚之後會用到的資料指令

命令	影響的層次	何時使用
<code>uv sync</code>	Python 套件環境	取得既有專案或同步依賴
<code>makemigrations</code>	產生結構變更檔	後續新增或修改 model
<code>migrate</code>	套用結構變更等 migration 操作	準備或更新資料庫
<code>seed_demo</code>	建立示範資料列	最後操作完成專案才使用

後三種命令接在 `uv run python manage.py` 後面。

`seed_demo` 是完成專案自訂的 command，**新建 Django 沒有它**。

自訂 model 與 ORM 留給後續章節；現在先完成最小網站。

2-20 啟動開發伺服器

```
uv run python manage.py runserver
```

終端機應出現類似訊息：

```
Starting development server at http://127.0.0.1:8000/
```

- `127.0.0.1` 指自己的電腦，`8000` 是服務連接埠（port）。
- 終端機保持運作，用瀏覽器開啟上面的網址。
- 修改 Python 檔通常會自動重載，新增檔案時必要時手動重啟。
- 按 `Ctrl+C` 停止。若 8000 被占用，可用 `runserver 8001`。

這是本機開發伺服器，尚未完成正式部署。

2-21 第一次 preview：看到 Django 歡迎頁

目前 `config/urls.py` 仍是產生的預設內容，只有 admin 路由。
在 `DEBUG=True` 的新專案根網址，應看到 Django 安裝成功頁。

- **步驟 A**：確認終端機仍在執行伺服器。
- **步驟 B**：在瀏覽器開啟 `http://127.0.0.1:8000/`。
- **步驟 C**：重新整理，觀察終端機的新紀錄。
- **步驟 D**：開啟瀏覽器開發者工具的 **Network**，再重新整理。

這裡的 preview 是瀏覽器向 Django 取得頁面。

直接雙擊 `.py` 或使用靜態 HTML 預覽工具，都不會執行 Django 網站。
稍後接入自己的路由後，根網址可能變成 404，屆時改測 `/hello/`。

2-22 啟動前後的檢查不能互相取代

檢查	能確認什麼	尚未證明什麼
check	部分設定、model 等結構問題	每個頁面都正確
migrate	migration 成功套用	已有示範資料
runserver	開發服務成功啟動	所有路由與業務功能正常
瀏覽器 preview	指定頁面可觀察的結果	所有輸入與邊界行為
test	已撰寫測試所涵蓋的行為	沒寫測試的功能

實作紀錄： 保存指令、畫面結果及一個遇到的錯誤。
接下來用 HTTP 解釋瀏覽器與終端機剛才發生了什麼。

2-23 第一次先跑 **check**

```
uv run python manage.py check
```

理想輸出：

```
System check identified no issues (0 silenced).
```

如果這一步就失敗，先不要跑 migrate/runserver；先處理 import/settings/URL 等問題。

2-24 啟動 server 後 Terminal 不會回 prompt

```
uv run python manage.py runserver
```

Terminal 會被 server process 佔住，這是正常的。

你要：

- 保留它看 request log / traceback
- 開第二個 terminal 跑 shell/test
- 停止 server：Ctrl+C

2-25 成功啟動長什麼樣？

你應該看到類似：

```
Starting development server at http://127.0.0.1:8000/  
Quit the server with CONTROL-C.
```

瀏覽器開：

```
http://127.0.0.1:8000/
```

不是把 URL 當成 terminal 指令。

2-26 觀念檢核：環境與啟動

1. `.venv` 與 `uv.lock` 各解決什麼問題？
2. 為什麼 `uv sync` 不會建立 Django 資料表？
3. `django>=6.1.1,<6.2` 接受哪些版本？
4. `uv run python manage.py seed_demo` 中，誰選環境？誰是專案入口？
5. 為什麼 `runserver` 成功還不代表可正式部署？

繳交： 自己的環境檢查結果，以及兩個完成專案的啟動紀錄。

配套手冊：[LearnBoard 原第 1 章](#)、[LearnMart 原第 1 章](#)。

手冊仍沿用原章號，請在完成本份練習後作對照。

第 2 章實作與離堂檢核

任務： 建立 config 與 pages，確認 app 註冊、migrate、check 及歡迎頁。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： LearnBoard 01 原第 1 章；LearnMart 01 原第 1 章。手冊保留原章號，對照表列出本課位置。

本份完成與後續

下一份：[02_HTTP路由與View](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。
- 原始教材與合併去向見 [來源索引](#)。